

# Question 1

Lets start by explaining the following code:

```
In [1]: # Part A
import numpy as np
from sklearn.datasets import fetch_california_housing
from sklearn.preprocessing import StandardScaler

A = np.array([[1,3,3],
              [2,2,-2],
              [-2,2,1]], dtype= float)
m, n = A.shape

Q = np.zeros((m,n))
R = np.zeros((n,n))

for j in range(n):
    v = A[:, j]
    for i in range(j):
        R[i,j] = np.dot(Q[:, i], A[:,j])
        v = v - R[i,j] * Q[:,i]
    R[j,j] = np.linalg.norm(v)
    Q[:,j] = v/R[j,j]

print(Q)
print(R)

# Part B
Q_np, R_np = np.linalg.qr(A)
print(Q_np)
print(R_np)

S = np.array([[-1,0,0],
              [0, -1, 0],
              [0,0,1]])

#Implementing the fix
Q_new = Q @ S
print(Q_new)
R_new = S @ R
print(R_new)

A_new = Q_new@R_new
print(A_new)
```

```

[[ 0.33333333  0.66666667  0.66666667]
 [ 0.66666667  0.33333333 -0.66666667]
 [-0.66666667  0.66666667 -0.33333333]]
[[ 3.  1. -1.]
 [ 0.  4.  2.]
 [ 0.  0.  3.]]
[[-0.33333333 -0.66666667  0.66666667]
 [-0.66666667 -0.33333333 -0.66666667]
 [ 0.66666667 -0.66666667 -0.33333333]]
[[-3. -1.  1.]
 [ 0. -4. -2.]
 [ 0.  0.  3.]]
[[-0.33333333 -0.66666667  0.66666667]
 [-0.66666667 -0.33333333 -0.66666667]
 [ 0.66666667 -0.66666667 -0.33333333]]
[[-3. -1.  1.]
 [ 0. -4. -2.]
 [ 0.  0.  3.]]
[[ 1.  3.  3.]
 [ 2.  2. -2.]
 [-2.  2.  1.]]

```

## Part A (COME BACK)

This code creates first the a matrix as an array in numpy, then it finds its dimensions. Next we set up a matrix of zeros for each corresponding Q and R which we will then fill in our double for loop. In the first half we are just telling which sections of our matrix we want to be filled like for example grabbing the j column to orthogonlaized in the second loop by dot product of our Q and A where then we subtract the current vertex we are wroking with. Finally over the first look we then normalize that vertex and then plug that into our R and then normailze the whole vertex in our Q.

## Part B

Done above

## Part C

It looks like we have oppostie signs for our first two rows and two columns. So baisacly in our matrix multiplication the signs are flipped which does not matter since if they are both negative or possitive we still end up with a possitive.

## Part D

The reason behind this is due to QE decomposition not being unique since we can always re-write our fomrlation in the following way.

$$Q = QS$$

$$R = SR$$

With  $S$  being a diagonal matrix of  $+$  or  $-$  1s. Where once we do  $(QS)(SR)$  it then becomes  $Q * I * R$  since  $S = S^{-1}$  which is simply  $QR$ . The reason  $S = S^{-1}$  is that when you multiply two diagonal matrices with  $+$  or  $-$  1s then we have them all become a possitive 1 since diagonal it then becomes the Identity matrix.

## Question 2

The following code is for the question

```
In [2]: A = np.array([[ -1, -3, 1],
                     [-2, 2, -2],
                     [ 2, -2, 0],
                     [ 4, 0, -2]])
Q_red, R_red = np.linalg.qr(A, mode='reduced')

Q_comp, R_comp = np.linalg.qr(A, mode='complete')
print(Q_red)
print(Q_comp)
print(R_red)
print(R_comp)

A_red = np.linalg.matmul(Q_red, R_red)
print(A_red, "Confirmed that it holds")

A_comp = np.linalg.matmul(Q_comp, R_comp)
print(A_comp, "Confirmed that it holds")
```

```

[[-0.2  0.8 -0.4]
 [-0.4 -0.4 -0.8]
 [ 0.4  0.4 -0.2]
 [ 0.8 -0.2 -0.4]]
[[-0.2  0.8 -0.4  0.4]
 [-0.4 -0.4 -0.8 -0.2]
 [ 0.4  0.4 -0.2 -0.8]
 [ 0.8 -0.2 -0.4  0.4]]
[[ 5. -1. -1.]
 [ 0. -4.  2.]
 [ 0.  0.  2.]]
[[ 5. -1. -1.]
 [ 0. -4.  2.]
 [ 0.  0.  2.]
 [ 0.  0.  0.]]
[[-1.00000000e+00 -3.00000000e+00  1.00000000e+00]
 [-2.00000000e+00  2.00000000e+00 -2.00000000e+00]
 [ 2.00000000e+00 -2.00000000e+00 -1.66533454e-16]
 [ 4.00000000e+00  2.88657986e-16 -2.00000000e+00]] Confirmed that it holds
[[-1.00000000e+00 -3.00000000e+00  1.00000000e+00]
 [-2.00000000e+00  2.00000000e+00 -2.00000000e+00]
 [ 2.00000000e+00 -2.00000000e+00 -1.66533454e-16]
 [ 4.00000000e+00  2.88657986e-16 -2.00000000e+00]] Confirmed that it holds

```

## Part A

Done above

## Part B

The difference between the two models is that in the reduced model we are getting rid of one of the rows for our R and one of the columns for our Q since if we have an  $m \times n$  matrix with  $m > n$  and we do the decomposition our bottom rows will be zero, thus it doesn't really matter and we can then take them out.

## Part C

Done above

## Question 3

```

In [3]: def back_substitution(R,b):
         n = R.shape[0]
         x = np.zeros(n, dtype = float)
         for i in range (n - 1, -1, -1):
             x[i] = (b[i] - np.dot(R[i, i+1:], x[i+1:]))/ R[i,i]
         return x

```

```

A = np.array([[2,1],
              [1,-1],
              [3,2],
              [4,1]])
y = np.array([1, 2, 3, 4], dtype=float)

# QR decomposition (reduced)
Q, R = np.linalg.qr(A, mode='reduced')

# Compute Q^T y
b = Q.T @ y

x_back = back_substitution(R,b)
res_back = np.linalg.norm(A @ x_back - y, 2)

x, residuals, rank, sv = np.linalg.lstsq(A,y)

print("x (lstsq):", x)
print("residuals:", residuals)
print("x_back (QR):", x_back)
print("res_back:", res_back)

```

```

x (lstsq): [ 1.16853933 -0.5505618 ]
residuals: [1.06741573]
x_back (QR): [ 1.16853933 -0.5505618 ]
res_back: 1.033158134235548

```

## Part A

First you count the number of rows within you matrix and create a matrix of the same size. Now we let our ith value in our list be equal to the solution for the  $Rx = b$  porblem where since we can split our matrix as the diagonals and the upper traingular we can then just sivide by these diagonal entires, followed by the numerator being the summation of of our off diagonal entires of R multiplied by x's and added, which are then subtracted by our bi'th entry

## Part B

Done above

## Part C

Done above, but simply it looks like the algorythm within `linalg.lstsq` is better at reducing the error, also I used euclidan norm since we are minimizing that norm itself by doing the least squares regression.

## Question 4

```
In [4]: data = fetch_california_housing()
X = data.data
y = data.target

scaler = StandardScaler()
X_design = scaler.fit_transform(X)

# QR decomposition (reduced)
Q, R = np.linalg.qr(X, mode='reduced')

# Compute Q^T y
b = Q.T @ y

w_back = back_substitution(R,b)

w_back

w, residuals, rank, sv = np.linalg.lstsq(X,y)

print("Predicted by NumPy Function")
print(w)
print("Predicted by Our Method")
print(w_back)
```

Predicted by NumPy Function

```
[ 5.13515163e-01  1.56511109e-02 -1.82528269e-01  8.65099057e-01
  7.79230657e-06 -4.69928985e-03 -6.39458199e-02 -1.63827177e-02]
```

Predicted by Our Method

```
[ 5.13515163e-01  1.56511109e-02 -1.82528269e-01  8.65099057e-01
  7.79230657e-06 -4.69928985e-03 -6.39458199e-02 -1.63827177e-02]
```

## Part A

Done above

## Part B

Done above

## Part C

Done above

## Question 5

```
In [19]: A = np.array([[4,1,-2],
                       [1,2,0],
```

```

        [-2,0,3]])

def eigval(A, iterations):
    A_k = A.astype(float).copy()
    m, n = A_k.shape

    def decomp(A_i):
        m, n = A_i.shape
        Q = np.zeros((m,n))
        R = np.zeros((n,n))

        for j in range(n):
            v = A_i[:, j].copy()
            for i in range(j):
                R[i,j] = np.dot(Q[:, i], A_i[:,j])
                v = v - R[i,j] * Q[:,i]
            R[j,j] = np.linalg.norm(v)
            Q[:,j] = v/R[j,j]
        return Q, R

    for _ in range(iterations):
        Q, R = decomp(A_k)
        A_k = R @ Q

    eigvals = np.diag(A_k)
    return eigvals

iterations = [1, 5, 8, 10, 100, 1000]

for iter_count in iterations:
    eiggg_ = eigval(A, iter_count)
    print("Our Function at " + str(iter_count) + " iterations")
    print(np.sort(eiggg_))
    print()

eigenvalues = np.linalg.eig(A)

print("Numpy Function")

print(np.sort(eigenvalues.eigenvalues))

```

Our Function at 1 iterations  
[1.31884058 2.0621118 5.61904762]

Our Function at 5 iterations  
[1.00035221 2.26762043 5.73202736]

Our Function at 8 iterations  
[1.00000259 2.26794669 5.73205072]

Our Function at 10 iterations  
[1.0000001 2.2679491 5.73205081]

Our Function at 100 iterations  
[1. 2.26794919 5.73205081]

Our Function at 1000 iterations  
[1. 2.26794919 5.73205081]

Numpy Function  
[1. 2.26794919 5.73205081]

## Part A

Done above

## Part B

Done above

## Part C

The accuracy is pretty much the exact same after 8 iterations which is pretty sweet as it converges pretty fast. Even after our first iterations we almost get there with only just a slight difference.